

INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY (ICDFA)

Digital Forensics Lab 4 - Steganography and Hidden Data Detection

Lab Title:	Steganography
Student Name:	Yusuf Oluwaseun Dada
Student ID:	2025/FWSD/11339
Course Code:	SBT-DF202
Course Name:	Computer and Digital Forensics

Instructor: Mr. Aminu Idris, AMCPN

Date: 05/09/2026

Introuction

What this lab actually teaches: steganography hides data inside another file (a picture that looks completely normal still contains a secret message), this is different from encryption, which scrambles data but makes it obviously present. A forensic examiner needs both skills: recognizing when a file might be hiding something, and knowing how to extract it.

Lab Setup: Tools and Workspace

```
(kali@kali)~[~/Desktop]
$ sudo apt update
sudo apt install -y steghide stegseek
which steghide stegseek sha256sum xxd diff strings file
[sudo] password for kali:
Get:1 https://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:2 https://kali.download/kali kali-rolling/main amd64 Packages [21.4 MB]
Get:3 https://kali.download/kali kali-rolling/main amd64 Contents (deb) [53.3 MB]
Get:4 https://kali.download/kali kali-rolling/contrib amd64 Packages [114 kB]
Get:5 https://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [264 kB]
Get:6 https://kali.download/kali kali-rolling/non-free amd64 Packages [183 kB]
Get:7 https://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [915 kB]
Fetched 76.2 MB in 1min 29s (856 kB/s)
85 packages can be upgraded. Run 'apt list --upgradable' to see them.
Installing:
  steghide stegseek

Installing dependencies:
  libmbedtls

Suggested packages:
  libmbedtls-dev mbedtls

Summary:
  Upgrading: 0, Installing: 3, Removing: 0, Not Upgrading: 85
  Download size: 355 kB
  Space needed: 1,133 kB / 50.5 GB available

Get:1 https://kali.download/kali kali-rolling/main amd64 libmbedtls amd64 2.5.8-8+b2 [71.8 kB]
Get:2 https://kali.download/kali kali-rolling/main amd64 steghide amd64 0.5.1+git20240220-2 [154 kB]
Get:3 https://kali.download/kali kali-rolling/main amd64 stegseek amd64 0.6+git20210910.ff677b9-2 [130 kB]
Fetched 355 kB in 2s (173 kB/s)
```

Session Actions Edit View Help

```
Unpacking libmbedtls:amd64 (2.5.8-8+b2)...
Selecting previously unselected package steghide.
Preparing to unpack ./steghide_0.5.1+git20240220-2_amd64.deb...
Unpacking steghide (0.5.1+git20240220-2)...
Selecting previously unselected package stegseek.
Preparing to unpack ./stegseek_0.6+git20210910.ff677b9-2_amd64.deb...
Unpacking stegseek (0.6+git20210910.ff677b9-2)...
Setting up libmbedtls:amd64 (2.5.8-8+b2)...
Setting up stegseek (0.6+git20210910.ff677b9-2)...
Setting up steghide (0.5.1+git20240220-2)...
Processing triggers for libc-bin (2.42-17)...
Processing triggers for man-db (2.13.1-1)...
Processing triggers for kali-menu (2026.3.2)...
Scanning processes ...
Scanning linux images ...

Running kernel seems to be up-to-date.

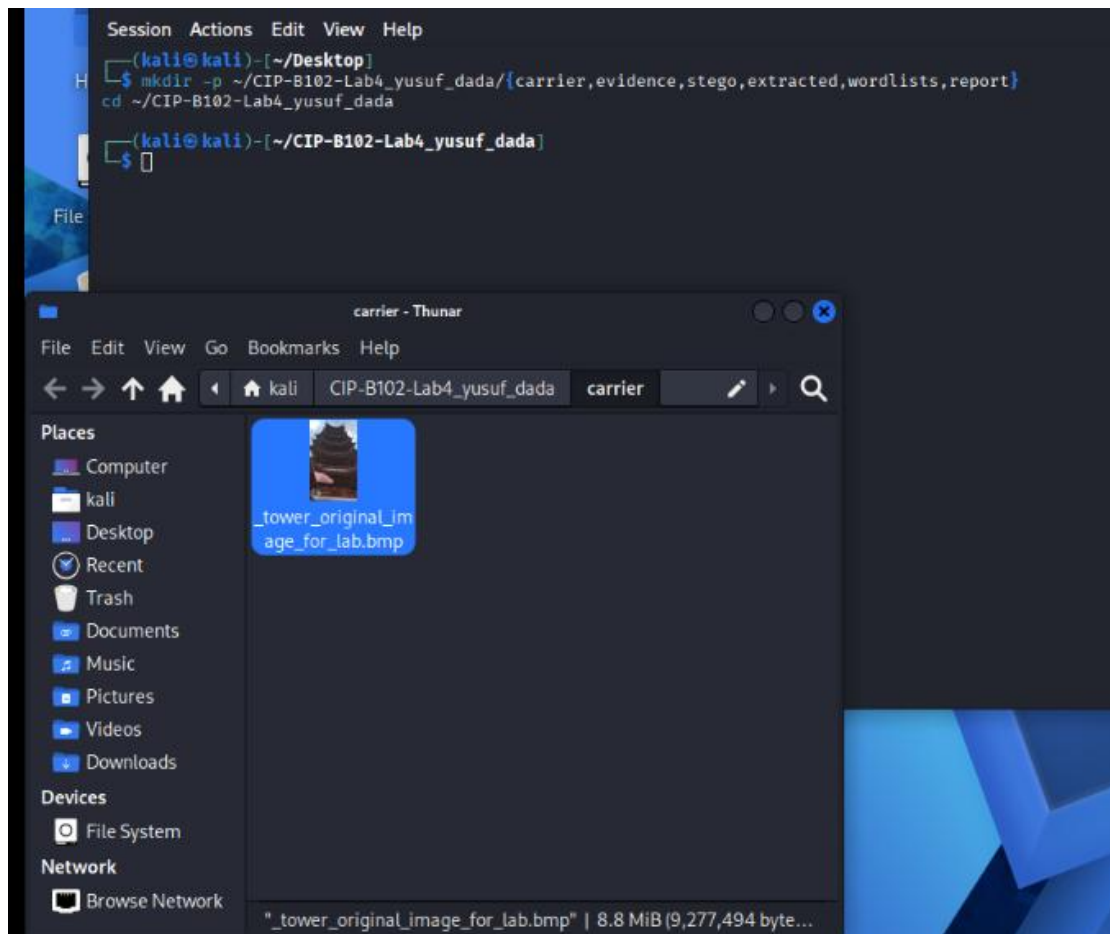
No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
/usr/bin/steghide
/usr/bin/stegseek
/usr/bin/sha256sum
/usr/bin/xxd
diff: aliased to diff --color=auto
/usr/bin/strings
/usr/bin/file
```

Lab Folder Creation



```
mkdir -p ~/CIP-B102-Lab4_yusuf_dada/{carrier,evidence,stego,extracted,wordlists,report}
cd ~/CIP-B102-Lab4_yusuf_dada
```

Part A - Prepare and Preserve the Carrier Image

The required BMP has been downloaded amoved “**Steganography Lab Carrier Image**” link into carrier/.

```
(kali@kali) [~/CIP-B102-Lab4_yusuf_dada]
$ cd ~/CIP-B102-Lab4_yusuf_dada/carrier
file _tower_original_image_for_lab.bmp
sha256sum _tower_original_image_for_lab.bmp | tee ../report/original_carrier_sha256.txt
_tower_original_image_for_lab.bmp: PC bitmap, Windows 3.x format, 1365 x 2265 x 24, resolution 2834 x 2834 px/m, cbSize 9277494, bits offset 54
6508bebb94bd8ec0833b9f85580cda4261d89632ddc89e7f9f7093b5e653c4db _tower_original_image_for_lab.bmp
```

```
cd ~/CIP-B102-Lab4_yusuf_dada/carrier
file _tower_original_image_for_lab.bmp
sha256sum _tower_original_image_for_lab.bmp | tee ../report/original_carrier_sha256.txt
6.txt
```

It shows a real BMP, plus the SHA-256 hash.

lesson: everything from here on works on this same file *read-only*, Steghide’s -cf (cover file) option doesn’t modify the original, it writes a **new** file via -sf (stego file). Still, don’t touch this original again after this point, just like a preserved baseline.

Part B - Create the Secret Evidence File

```
(kali@kali) [~/CIP-B102-Lab4_yusuf_dada/carrier]
$ cd ~/CIP-B102-Lab4_yusuf_dada/evidence
cat > secret.txt <<'EOF'
Full Name: Yusuf Oluwseun Dada
Message: This is a controlled evidence note for the CIP-B102 steganography lab.
Confirmation: This file was created for the ICDFA lab.
EOF
cat secret.txt
sha256sum secret.txt | tee ../report/secret_txt_sha256.txt
Full Name: Yusuf Oluwseun Dada
Message: This is a controlled evidence note for the CIP-B102 steganography lab.
Confirmation: This file was created for the ICDFA lab.
01ab57969e94cb5a9c352cad0b80a67ca8669c7f0c8a6a953f025241527ce963 secret.txt
```

```
cd ~/CIP-B102-Lab4_yusuf_dada/evidence
cat > secret.txt <<'EOF'
Full Name: Yusuf Oluwseun Dada
Message: This is a controlled evidence note for the CIP-B102 steganography lab.
Confirmation: This file was created for the ICDFA lab.
EOF
cat secret.txt
sha256sum secret.txt | tee ../report/secret_txt_sha256.txt
```

Part C - Embed the Hidden File with Steghide

```
$ cd ~/CIP-B102-Lab4_yusuf_dada
steghide embed \
  -ef evidence/secret.txt \
  -cf carrier/_tower_original_image_for_lab.bmp \
  -sf stego/_tower_stego_lab4.bmp \
  -p 1234
embedding "evidence/secret.txt" in "carrier/_tower_original_image_for_lab.bmp" ... done
writing stego file "stego/_tower_stego_lab4.bmp" ... done
```

```
cd ~/CIP-B102-Lab4_yusuf_dada
steghide embed \
  -ef evidence/secret.txt \
  -cf carrier/_tower_original_image_for_lab.bmp \
  -sf stego/_tower_stego_lab4.bmp \
  -p 1234
```

Steghide prompt for confirmation and then report success as seen the screenshot “Done”

```
(kali@kali)-[~/CIP-B102-Lab4_yusuf_dada]
$ sha256sum stego/_tower_stego_lab4.bmp | tee report/stego_image_sha256.txt
ls -lh carrier/_tower_original_image_for_lab.bmp stego/_tower_stego_lab4.bmp
e7cc9157913a2bc9346153efb1beac355b25cff5edcda42fca5c2bcfeb6b98dc  stego/_tower_stego_lab4.bmp
-rwxrwx-rw- 1 kali kali 8.9M Sep  3 12:46 carrier/_tower_original_image_for_lab.bmp
-rw-rw-r-- 1 kali kali 8.9M Sep  3 20:15 stego/_tower_stego_lab4.bmp
```

```
sha256sum stego/_tower_stego_lab4.bmp | tee report/stego_image_sha256.txt
ls -lh carrier/_tower_original_image_for_lab.bmp stego/_tower_stego_lab4.bmp
```

The embed command completing successfully, plus the stego image’s hash.

Note the sizes: a small text file embedded via LSB substitution often barely changes the BMP’s file size at all (LSB techniques modify pixel data in place rather than appending data), which is exactly what makes this method hard to detect visually.

Part D - Extract and Verify the Hidden Evidence

```
(kali@kali)-[~/CIP-B102-Lab4_yusuf_dada]
$ mkdir -p extracted
steghide extract \
  -sf stego/_tower_stego_lab4.bmp \
  -xf extracted/extracted_secret.txt \
  -p 1234
wrote extracted data to "extracted/extracted_secret.txt".
```

```
mkdir -p extracted
steghide extract \
  -sf stego/_tower_stego_lab4.bmp \
  -xf extracted/extracted_secret.txt \
  -p 1234
```

It shows successful extraction message.

Verify it's identical to the original:

```
(kali㉿kali)-[~/CIP-B102-Lab4_yusuf_dada]
$ diff evidence/secret.txt extracted/extracted_secret.txt 86 echo "MATCH: files are identical"
sha256sum evidence/secret.txt extracted/extracted_secret.txt
MATCH: files are identical
01ab57969e94cb5a9c352cad0b80a67ca8669c7f0c8a6a953f025241527ce963 evidence/secret.txt
01ab57969e94cb5a9c352cad0b80a67ca8669c7f0c8a6a953f025241527ce963 extracted/extracted_secret.txt
```

```
diff evidence/secret.txt extracted/extracted_secret.txt && echo "MATCH: files are id
entical"
```

```
sha256sum evidence/secret.txt extracted/extracted secret.txt
```

Part E - Compare Original and Stego Images

```

L-$ cd ~/CIP-B102-Lab4_yusuf_dada

echo "=== File sizes ==="
ls -lh carrier/_tower_original_image_for_lab.bmp stego/_tower_stego_lab4.bmp

echo "=== SHA-256 hashes ==="
sha256sum carrier/_tower_original_image_for_lab.bmp stego/_tower_stego_lab4.bmp

echo "=== First 64 bytes (hex) ==="
xxd carrier/_tower_original_image_for_lab.bmp | head -n 4
xxd stego/_tower_stego_lab4.bmp | head -n 4

echo "=== Readable strings ==="
strings carrier/_tower_original_image_for_lab.bmp | head -n 10
strings stego/_tower_stego_lab4.bmp | head -n 10

=== File sizes ===
-rwxrw-rw- 1 kali kali 8.9M Sep  3 12:46 carrier/_tower_original_image_for_lab.bmp
-rw-rw-r-- 1 kali kali 8.9M Sep  3 20:15 stego/_tower_stego_lab4.bmp

=== SHA-256 hashes ===
6508bebb94bd8ec0833b9f85580cda4261d89632ddc89e7f9f7093b5e653c4db  carrier/_tower_original_image_for_lab.bmp
e7cc9157913a2bc9346153efb1beac355b25cff5edcda42fca5c2bcfeb6b98dc  stego/_tower_stego_lab4.bmp

=== First 64 bytes (hex) ===
00000000: 424d 3690 8d00 0000 0000 3600 0000 2800  BM6.....6...(.
00000010: 0000 5505 0000 d908 0000 0100 1800 0000  ..U.....
00000020: 0000 0000 0000 120b 0000 120b 0000 0000  .....5 #8. 8.
00000030: 0000 0000 0000 1e22 3520 2338 1e20 381f  .....5 #8. 8.
00000000: 424d 3690 8d00 0000 0000 3600 0000 2800  BM6.....6...(.
00000010: 0000 5505 0000 d908 0000 0100 1800 0000  ..U.....
00000020: 0000 0000 0000 120b 0000 120b 0000 0000  .....
00000030: 0000 0000 0000 1e22 3520 2338 1e20 381f  .....5 #8. 8.

=== Readable strings ===
"5 #8

```

[illegible]

```
cd ~/CIP-B102-Lab4 yusuf dada
```

```
echo "=== File sizes ==="
```

ls -lh carrier/ tower original image for lab.bmp stego/ tower stego lab4.bmp

```
echo "=== SHA-256 hashes ==="
```

sha256sum carrier/ tower original image for lab.bmp stego/ tower stego lab4.bm

p

```
echo "=== First 64 bytes (hex) ==="
xxd carrier/_tower_original_image_for_lab.bmp | head -n 4
xxd stego/_tower_stego_lab4.bmp | head -n 4

echo "=== Readable strings ==="
strings carrier/_tower_original_image_for_lab.bmp | head -n 10
strings stego/_tower_stego_lab4.bmp | head -n 10
```

The two images are visually indistinguishable because Steghide's LSB (least significant bit) substitution technique only changes the least significant bit of selected pixel color values, a change so small it's imperceptible to the human eye and doesn't alter the image's dimensions or visible structure. However, the underlying binary data **is** different at the bit level, which is exactly why the SHA-256 hashes differ completely even though the images look identical when opened. This is precisely why hash comparison, not visual inspection, is the reliable way to detect whether a file has been altered.

Part F - Controlled Password Dictionary Test (StegSeek)



```
(kali@kali)-[~/CIP-B102-Lab4_yusuf_dada]
$ cd ~/CIP-B102-Lab4_yusuf_dada/wordlists
echo "1234" > lab4_wordlist.txt
cat lab4_wordlist.txt

cd ..
stegseek stego/_tower_stego_lab4.bmp wordlists/lab4_wordlist.txt
1234
StegSeek 0.6 - https://github.com/RickdeJager/StegSeek

[i] Found passphrase: "1234"
[i] Original filename: "secret.txt".
[i] Extracting to "_tower_stego_lab4.bmp.out".
```

```
cd ~/CIP-B102-Lab4_yusuf_dada/wordlists
echo "1234" > lab4_wordlist.txt
cat lab4_wordlist.txt
```

```
cd ..
stegseek stego/_tower_stego_lab4.bmp wordlists/lab4_wordlist.txt
```

StegSeek finding the password and successfully extracting, it shows report like
"Found passphrase: '1234'"

Part G - Independent Student Task (do this fully on your own inputs)

This repeats the entire workflow above, but with my **own** file, password, and wordlist this is the part that proves you understand the process rather than just following steps.

G1. Create your own hidden evidence file

```
(kali@kali)-[~/CIP-B102-Lab4_yusuf_dada]
$ cd ~/CIP-B102-Lab4_yusuf_dada/evidence
cat > yusuf_dada_hidden_note.txt <<'EOF'
steganography is the practice of data hiding, it can be text, images or files inside another ordinary files or unnoticeable files. It matters in digital forensics because criminals may use it to transmit evidence and investigators must detect the hidden payload to uncover illicit activities and to maintain the integrity of digital evidence in investigation
EOF
cat yusuf_dada_hidden_note.txt
steganography is the practice of data hiding, it can be text, images or files inside another ordinary files or unnoticeable files. It matters in digital forensics because criminals may use it to transmit evidence and investigators must detect the hidden payload to uncover illicit activities and to maintain the integrity of digital evidence in investigation
EOF
```

cd ~/CIP-B102-Lab4_yusuf_dada/evidence

cat > yusuf_dada_hidden_note.txt <<'EOF'

steganography is the practice of data hiding, it can be text, images or files inside another ordinary files or unnoticeable files. It matters in digital forensics because criminals may use it to transmit evidence and investigators must detect the hidden payload to uncover illicit activities and to maintain the integrity of digital evidence in investigation

EOF

G2. Embed it with a password of your own choosing

```
(kali@kali)-[~/CIP-B102-Lab4_yusuf_dada/evidence]
$ cd ~/CIP-B102-Lab4_yusuf_dada
steghide embed \
  -ef evidence/yusuf_dada_hidden_note.txt \
  -cf carrier/_tower_original_image_for_lab.bmp \
  -sf stego/yusuf_dada_stego.bmp \
  -p 12345
embedding "evidence/yusuf_dada_hidden_note.txt" in "carrier/_tower_original_image_for_lab.bmp" ... done
writing stego file "stego/yusuf_dada_stego.bmp" ... done
```

cd ~/CIP-B102-Lab4_yusuf_dada

steghide embed \

-ef evidence/yusuf_dada_hidden_note.txt \

-cf carrier/_tower_original_image_for_lab.bmp \

-sf stego/yusuf_dada_stego.bmp \

-p 12345

I Screenshotted the successful embed.

G3. Extract it again

```
(kali@kali)-[~/CIP-B102-Lab4_yusuf_dada]
$ steghide extract \
  -sf stego/yusuf_dada_stego.bmp \
  -xf extracted/yusuf_dada_extracted.txt \
  -p 12345
wrote extracted data to "extracted/yusuf_dada_extracted.txt".
```

steghide extract \

-sf stego/yusuf_dada_stego.bmp \

-xf extracted/yusuf_dada_extracted.txt \

-p 12345

G4. Verify integrity with hashes

```
(kali@kali)-[~/CIP-B102-Lab4_yusuf_dada]
$ diff evidence/yusuf_dada_hidden_note.txt extracted/yusuf_dada_extracted.txt 56 echo "MATCH"
sha256sum evidence/yusuf_dada_hidden_note.txt extracted/yusuf_dada_extracted.txt
MATCH
6aa46004a41cfedd4df8d66a401c9614ca7d43e91aa707dce2f83ff42bd268e2 evidence/yusuf_dada_hidden_note.txt
6aa46004a41cfedd4df8d66a401c9614ca7d43e91aa707dce2f83ff42bd268e2 extracted/yusuf_dada_extracted.txt
```

```
diff evidence/yusuf_dada_hidden_note.txt extracted/yusuf_dada_extracted.txt && echo "MATCH"
sha256sum evidence/yusuf_dada_hidden_note.txt extracted/yusuf_dada_extracted.txt
```

Screenshot the match confirmation.

G5. Create a wordlist containing your chosen password

```
(kali@kali)~[~/CIP-B102-Lab4_yusuf_dada]
$ cd wordlists
echo "12345" > yusuf_dada_wordlist.txt
cd ..
```

```
cd wordlists
echo "12345" > yusuf_dada_wordlist.txt
cd ..
```

G6. Test the recovery workflow with StegSeek

```
(kali@kali)~[~/CIP-B102-Lab4_yusuf_dada]
$ stegseek stego/yusuf_dada_stego.bmp wordlists/yusuf_dada_wordlist.txt
StegSeek 0.6 - https://github.com/RickdeJager/StegSeek

[i] Found passphrase: "12345"
[i] Original filename: "yusuf_dada_hidden_note.txt".
[i] Extracting to "yusuf_dada_stego.bmp.out".
```

```
stegseek stego/yusuf_dada_stego.bmp wordlists/yusuf_dada_wordlist.txt
```

Final Step - Submission analysis in forensics

```
$ cd ~/CIP-B102-Lab4_yusuf_dada
zip -r CIP-B102-Lab4_yusuf_dada_Submission.zip \
evidence stego extracted wordlists report
ls -lh CIP-B102-Lab4_yusuf_dada_Submission.zip
adding: evidence/ (stored 0%)
adding: evidence/yusuf_dada_hidden_note.txt (deflated 40%)
adding: evidence/secret.txt (deflated 19%)
adding: stego/ (stored 0%)
adding: stego/yusuf_dada_stego.bmp (deflated 55%)
adding: stego/_tower_stego_lab4.bmp (deflated 55%)
adding: extracted/ (stored 0%)
adding: extracted/extracted_secret.txt (deflated 19%)
adding: extracted/yusuf_dada_extracted.txt (deflated 40%)
adding: wordlists/ (stored 0%)
adding: wordlists/yusuf_dada_wordlist.txt (stored 0%)
adding: wordlists/lab4_wordlist.txt (stored 0%)
adding: report/ (stored 0%)
adding: report/secret_txt_sha256.txt (deflated 12%)
adding: report/original_carrier_sha256.txt (stored 0%)
adding: report/stego_image_sha256.txt (deflated 12%)
-rw-rw-r-- 1 kali kali 7.9M Sep  4 17:04 CIP-B102-Lab4_yusuf_dada_Submission.zip
```

```
cd ~/CIP-B102-Lab4_yusuf_dada
zip -r CIP-B102-Lab4_yusuf_dada_Submission.zip \
evidence stego extracted wordlists report
ls -lh CIP-B102-Lab4_yusuf_dada_Submission.zip
```